



西安电子科技大学  
XIDIAN UNIVERSITY

《操作系统》

# 第三章 存储管理



人工智能学院

## 七、内存扩充 (2) 虚拟存储

**OS 是魔术师**

**由小变大、由少变多**

## 七、内存扩充（2） 虚拟存储

- 为了在内存空间运行超过内存总容量的大作业
  - 或者同时运行大量作业
  - 解决的方法是从逻辑上扩充内存容量
  - 这就是虚拟存储技术所要解决的主要问题
- 
- 实现虚拟存储器要解决：
    - 程序部分运行可以吗？
    - 发现程序不在内存时，如何将其装入后继续运行？
    - 内存无空间时怎么办？

# 七、内存扩充 (2) 虚拟存储

## 程序执行规律

- ❖ 程序中不是每一条指令都会在程序的一次运行过程中执行到。
  - 错误处理子程序
  - 条件语句(**if...else...**)
- ❖ 程序中有的指令可能只执行一次
  - 程序的初始化部分
- ❖ 程序执行的局部性原理
  - 在一段时间内，作业一般不会执行到所有程序的指令，也不会存取绝大部分数据，执行的代码和要存取的数据往往集中在某些区域中(例如一个循环、一个数组)。





# 七、内存扩充（2） 虚拟存储

## 【程序的局部性原理】

- ❖ 在程序装入时，不必一次将其全部读入到内存，而只需将当前需要执行的某些区域读入到内存，然后程序开始执行。在程序执行过程中，如果需执行的指令或访问的数据尚未在内存，则由处理器通知操作系统将相应的区域调入内存，然后继续执行。
- 在一段时间内，程序的执行仅局限于某个部分；相应地，它所访问的存储空间也局限于某个区域内。
  - 程序在执行时，除了少部分的转移和过程调用指令外，**大多数仍是顺序执行的**。
  - **子程序调用**将会使程序的执行由一部分内存区域转至另一部分区域。但在大多数情况下，过程调用的深度都不超过 5 层。
  - 程序中存在许多**循环结构**，循环体中的指令被多次执行。
  - 程序中还包括许多对**数据结构的处理**，如对连续的存储空间——数组的访问，往往局限于很小的范围内。

## 七、内存扩充 (2) 虚拟存储

- Virtual memory is a technique that allows the execution of processes that may not be completely in memory.

(**虚拟内存**是一种允许进程部分装入内存就可以执行的技术)

- principle of locality 局部性原理：时间局部性，空间局部性
- Only part of the program needs to be in memory for execution  
(只有运行的部分程序需要在内存中)

优点：

- ❖ 程序的大小可以突破内存容量限制，使得用户感觉到系统好像提供了一个容量极大的“主存”。
- ❖ 内存中容纳更多程序并发执行，系统效率得到提升。

## 七、内存扩充（2） 虚拟存储

### □ 时间局部性：

- 由于程序中存在着**大量的循环操作**
- 某条指令一旦执行，则不久该指令可能再次被执行；
- 某个存储单元被访问，则不久该存储单元可能再次被访问。

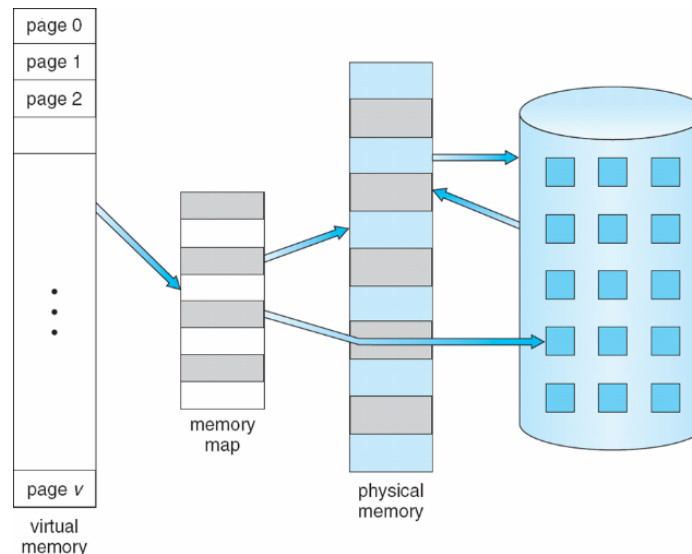
### □ 空间局部性：

- 由于**程序的顺序执行**
- 一旦程序访问了某个存储单元，则其附近的存储单元也最有可能被访问。即程序在一段时间内所访问的地址，可能集中在一定的范围内

## 七、内存扩充（2） 虚拟存储

### Virtual Memory That is Larger Than Physical Memory

- Logical address space can therefore be much larger than physical address space（逻辑地址空间能够比物理地址空间大）。
  - Need to allow pages to be swapped in and out（必须允许页面能够被换入和换出）。



## 七、内存扩充 (2) 虚拟存储

### 虚拟存储器的特征

- **离散性**: 在内存分配时采用离散的方式, 是虚拟存储器的最基本的特征。
- **多次性**: 一个作业被分成多次调入内存运行, 即在作业运行时没有必要将其全部装入, 只须将当前要运行的那部分程序和数据装入内存即可。是虚拟存储器最重要的特征。
- **对换性**: 作业运行过程中信息在内存和外存的对换区之间换进、换出。
- **虚拟性**: 从逻辑上扩充内存容量, 使用户所看到的内存容量远大于实际内存容量。

## 七、内存扩充 (2) 虚拟存储

### Background

- Virtual memory can be implemented via (虚拟内存能够通过以下方法来实现) :
  - Demand paging (请求页式)
  - Demand segmentation (请求段式)

## 七、内存扩充（2） 虚拟存储

### Demand Paging

- 在分页系统的基础上，增加了请求调页功能、页面置换功能，从而支持形成虚拟存储系统
- 需解决：
  - 取页--将哪部分装入内存
  - 置页--将调入的页放在什么地方
  - 淘汰--内存不足时，将哪些页换出内存

## 七、内存扩充 (2) 虚拟存储

### Demand Paging

- Bring a page into memory only when it is needed (只有一个页需要的时候才把它装入内存) .
  - Less I/O needed (需要很少的I/O)
  - Less memory needed (需要很少的内存)
  - Faster response (快速响应)
  - More users (多用户)
- Hardware Support
  - invalid reference (无效的访问)  $\Rightarrow$  abort (中止)
  - not-in-memory (不在内存)  $\Rightarrow$  bring to memory (换入内存)



## 七、内存扩充 (2) 虚拟存储

### Page table for demand paging

- 在分页的页表机制上形成
- 增加若干信息项，供程序(数据)在换进、换出时参考

| 页号 | 物理块号 | 状态位P | 访问字段A | 修改位M | 外存地址 |
|----|------|------|-------|------|------|
|    |      |      |       |      |      |

- ✓ **状态位 (存在位P)**：指示该页是否已调入内存。
- ✓ **访问字段A**：记录本页在一段时间内被访问的次数，或最近已有多长时间未被访问，提供给置换算法选择换出页面时参考。
- ✓ **修改位M**：表示该页在调入内存后是否被修改过。
- ✓ **外存地址**：指出该页在外存上的地址，供调入该页时使用。

## 七、内存扩充（2） 虚拟存储

### Page fault（缺页中断）

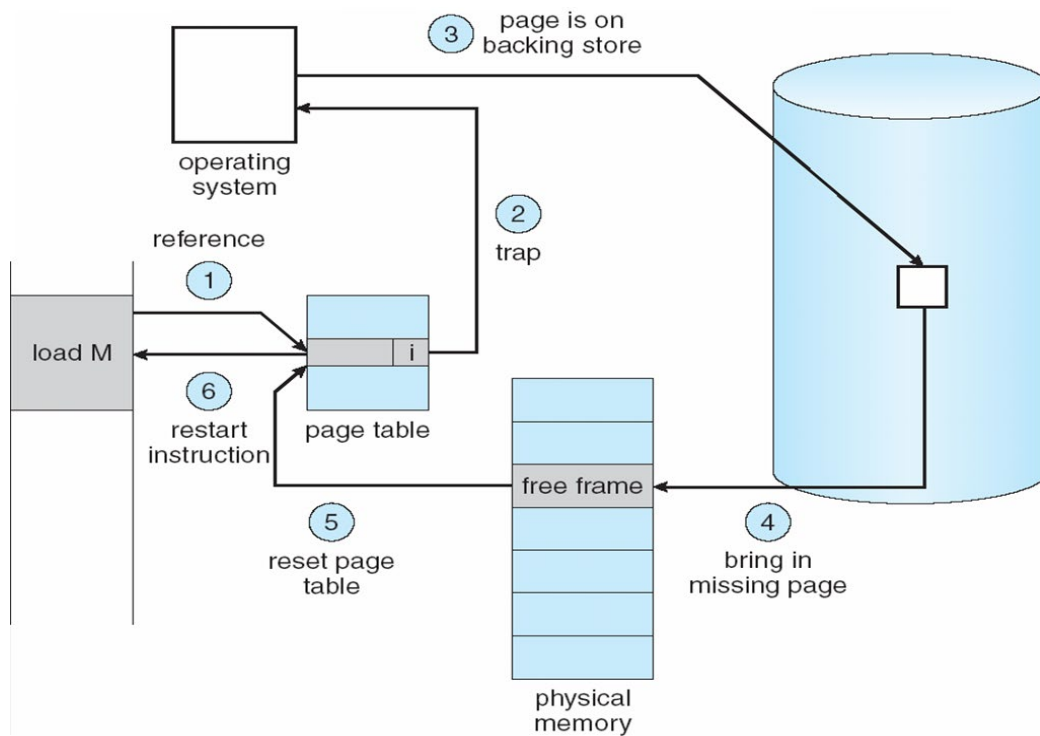
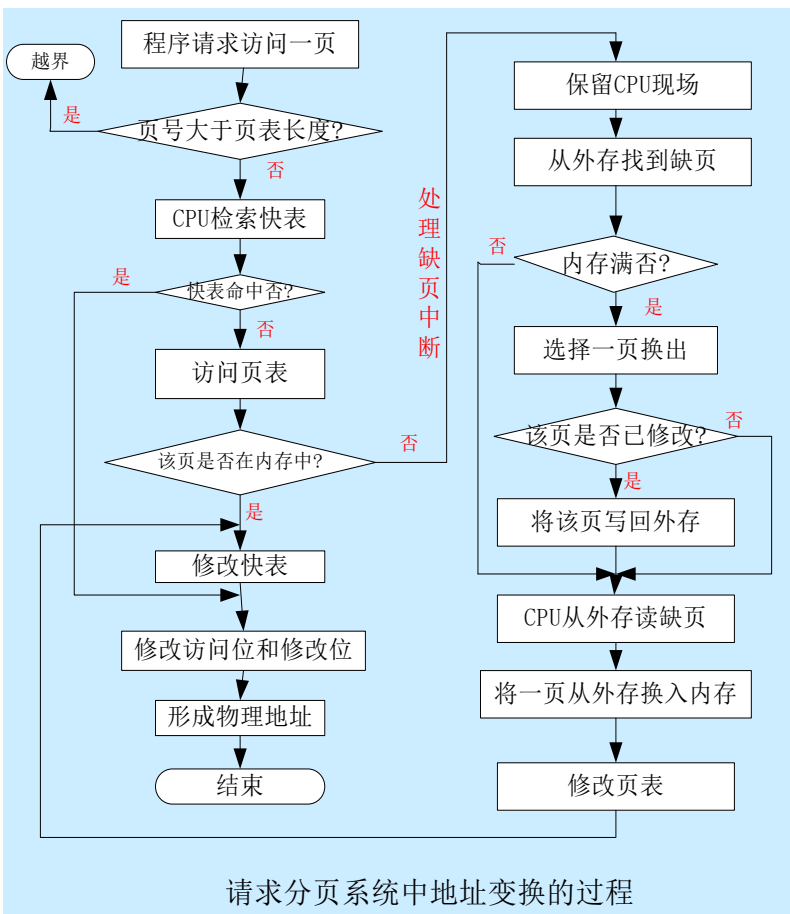
- 在分页系统地址变换机制的基础上，增加了：
  - 产生和处理缺页中断
  - 页面置换功能等
- ✓ 每当所要访问的页面不在内存时，便产生——**缺页中断**(page fault)，请求OS将所缺页调入内存。
- ✓ 与一般中断的**主要区别**在于：
  - 缺页中断在**指令执行期间**产生和处理中断信号，而一般中断在一条指令执行完后检查和处理中断信号。
  - 缺页中断返回到该指令的开始**重新执行该指令**，而一般中断返回到该指令的下一条指令执行。
  - 一条指令在执行期间，**可能产生多次缺页中断**。

## 七、内存扩充 (2) 虚拟存储

### Steps in handling a page fault

- **合法性检查**: 查找页表来确定此次地址访问是否合法
- **缺页处理**: 如果不合法, 则中止该进程; 否则如果是发生了缺页, 则需要将其调入内存
- **分配物理块**: 找到一个空闲物理块
- **磁盘读页**: 启动磁盘, 把该页读入内存
- **更新页表**: 读磁盘结束后, 修改页表以指出该页已在内存中
- **重执行指令**: 重新开始执行刚才发生缺页中断的指令, 这时它可以访问刚才调入的页

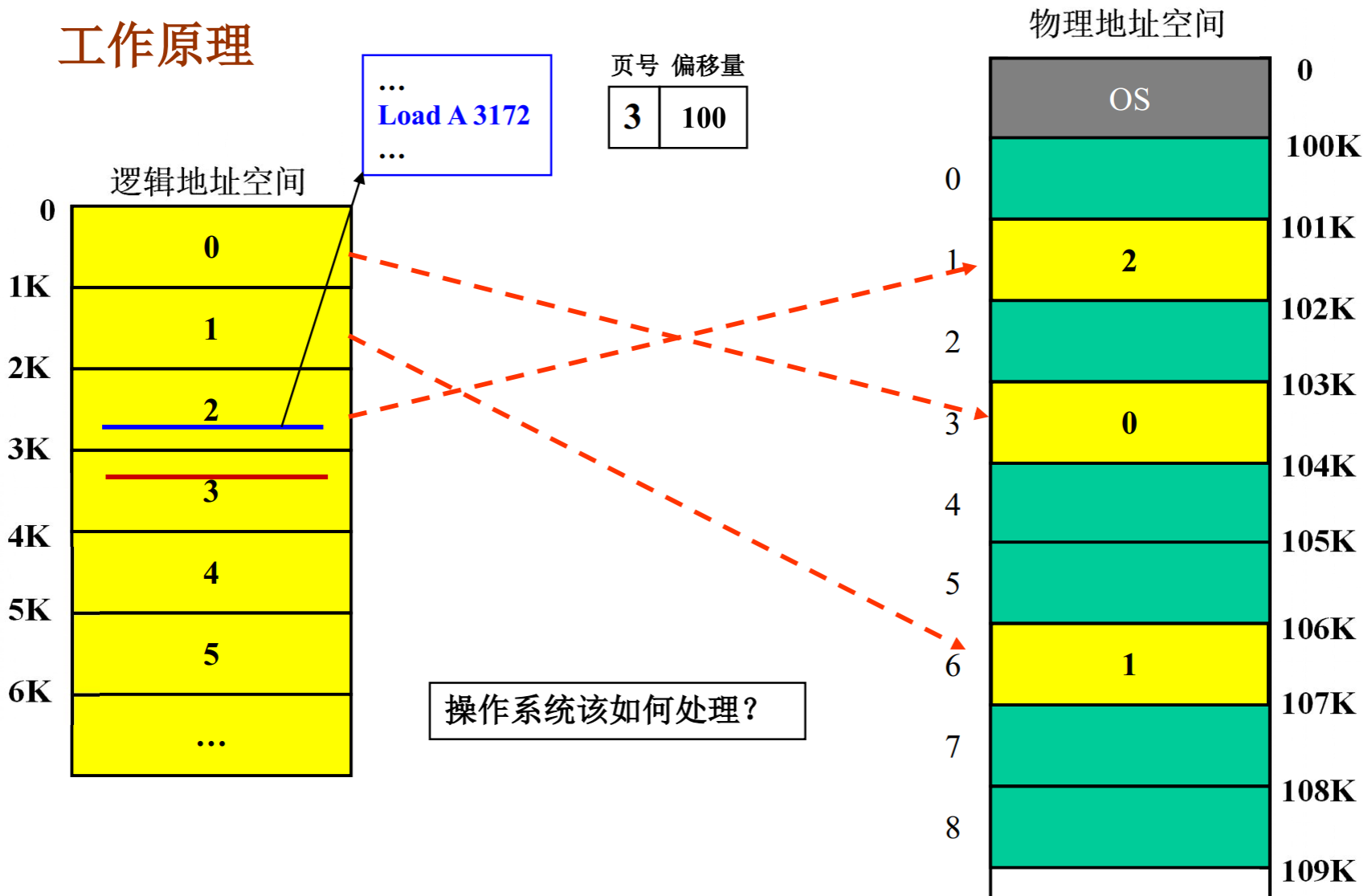
# 七、内存扩充 (2) 虚拟存储



# 七、内存扩充 (2) 虚拟存储



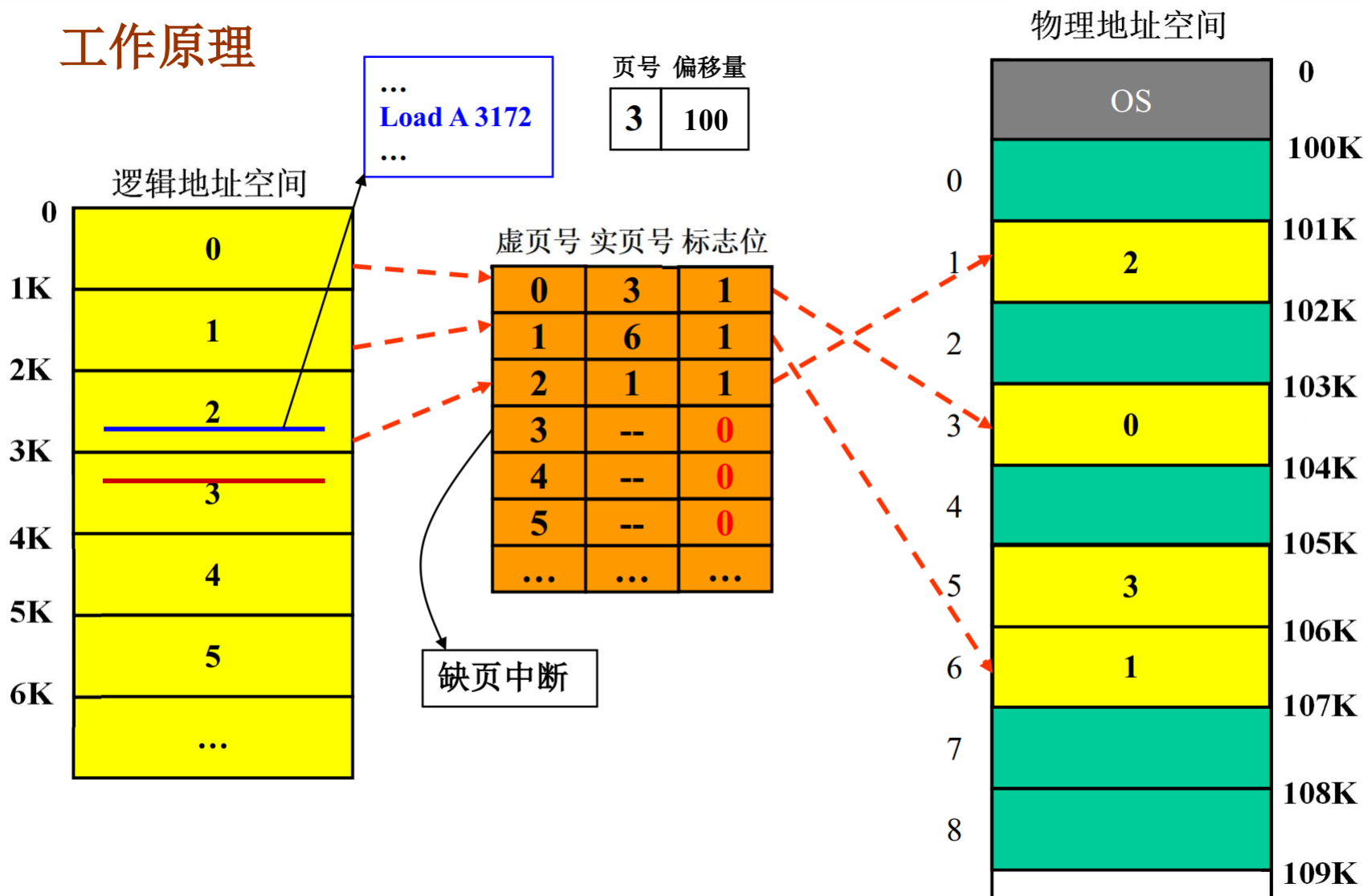
## 工作原理



# 七、内存扩充 (2) 虚拟存储



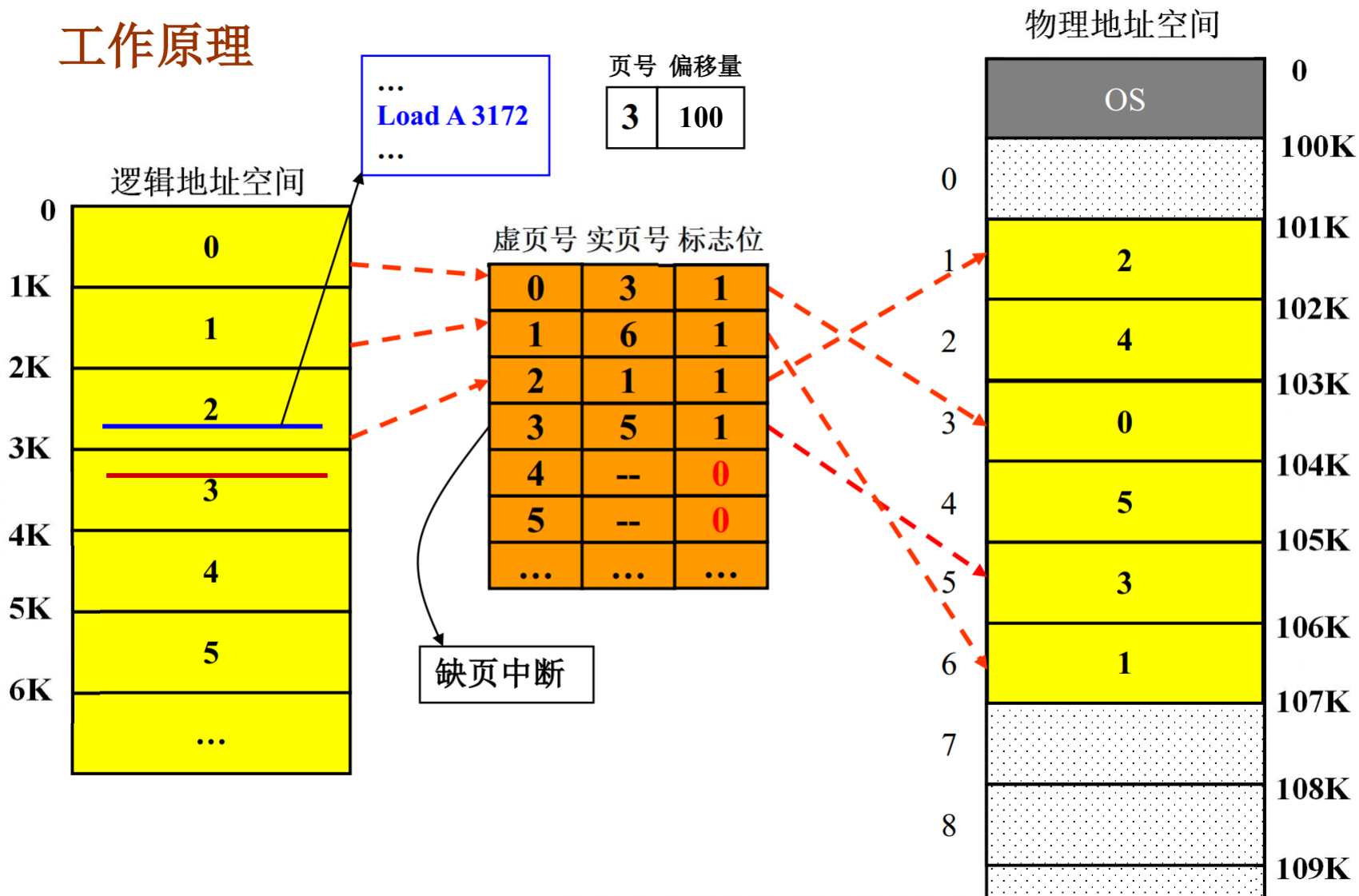
## 工作原理



# 七、内存扩充 (2) 虚拟存储



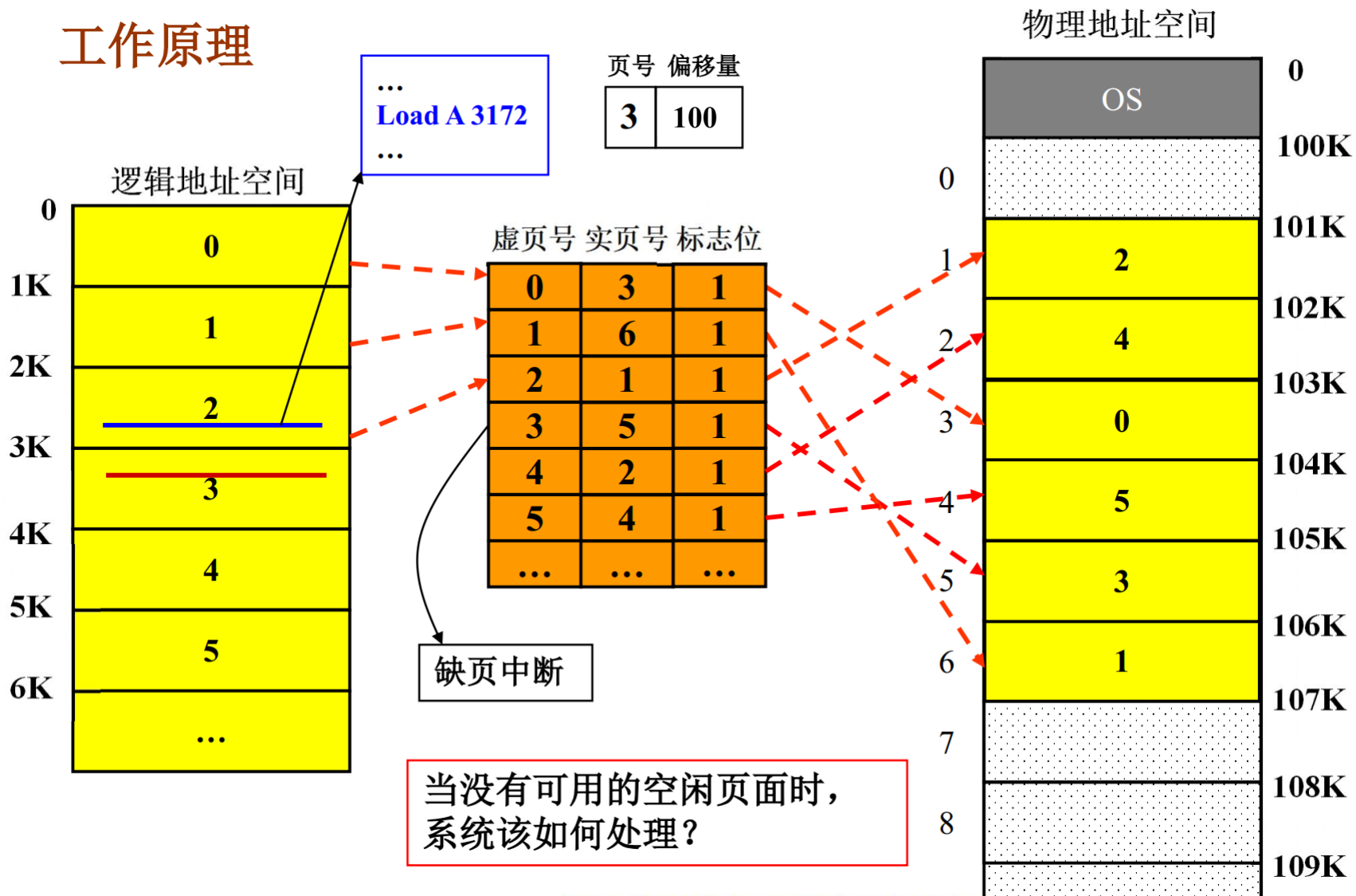
## 工作原理



# 七、内存扩充 (2) 虚拟存储



## 工作原理





## 七、内存扩充 (2) 虚拟存储

### What happens if there is no free frame?

- ▣ Page replacement – find some page in memory, but not really in use, swap it out

(页置换—找到内存中并没有使用的某个页，换出)

- ✓ Algorithm (算法)

- ✓ Performance (性能)

- want an algorithm which will result in minimum number of page faults

(找出一个导致最小缺页数的算法)

# 七、内存扩充（2） 虚拟存储

## 页面淘汰算法

- ❖ 功能：在可用页面不足时，确定内存中哪个物理页面将被淘汰。
- ❖ 出发点：希望把未来不再使用的或者短时期内较少使用的页面调出。
- ❖ 常见的页面淘汰算法
  - 先进先出页面置换算法（FIFO）
  - 最近最久未使用置换算法（LRU）
  - 最佳置换算法（OPT）
  - 时钟置换算法（CLOCK）

## Basic Page Replacement

- Find the location of the desired page on disk
- Find a free frame:
  - If there is a free frame, use it
  - If there is no free frame, use a page replacement algorithm to select a **victim** frame
- Bring the desired page into the (newly) free frame; update the page and frame tables

# 七、内存扩充 (2) 虚拟存储

## 页面调入策略

- 调入页面的时机

- **预调页策略**（为能使进程运行，事先需将一部分要执行的程序和数据调入内存）：
  - 主动的页面调入策略，即把那些预计很快会被访问的程序或数据所在的页面，预先调入内存。
  - 预测的准确率不高（50%），主要用于进程的首次调入。  
也有的系统将预调页策略用于请求调页的辅助
- **请求调页策略**：
  - 当进程在运行中发生缺页时，由系统将缺页调入内存
  - 目前虚拟存储器系统大多采用此策略
  - 在调页时须花费较大的系统开销，如需频繁启动磁盘I/O

# 七、内存扩充 (2) 虚拟存储

## 页面调入策略

- 从何处调入页面
  - 在虚拟存储系统中，外存（硬盘）被分成两部分：文件区和对换区。
  - 对换区（连续分配）的磁盘I/O速度比文件区（离散分配）要高。

## 七、内存扩充 (2) 虚拟存储

### Performance of Demand Paging

- 发生缺页时会导致以下步骤的发生:
  1. 陷入OS
  2. 保存该用户寄存器和进程状态
  3. 确定该中断是一个缺页中断
  4. 检查该页面引用是合法的并确定该页在磁盘上的位置
  5. 将该页从磁盘读入一个空闲物理块:
    - 在磁盘等待队列中等待直到该请求被处理
    - 等待设备寻道延迟
    - 将该块从磁盘传送至内存
  6. 为了提高CPU利用率, 将CPU分派给其他进程使用

## 七、内存扩充 (2) 虚拟存储

### Performance of Demand Paging

7. 磁盘I/O完成, 产生中断
8. 保存正在执行进程的现场信息 (如果第6步执行了)
9. 确定中断来自于磁盘
10. 修改页表以示所缺的页已进入内存
11. 等待CPU再次分派给这个进程
12. 恢复该进程的现场信息, 包括寄存器、进程状态、页表等,  
恢复执行

## 七、内存扩充 (2) 虚拟存储

### Performance of Demand Paging

- 以上步骤并不是在任何情况下都会发生的
- 这里主要的动作是：
  - 处理缺页中断
  - 从磁盘读入所需的页
  - 重新开始被中断的进程
- 其中最大的一部分时间开销为从磁盘读入所需的页

## 七、内存扩充 (2) 虚拟存储

### Performance of Demand Paging

#### 缺页率

- 假定作业  $J_i$  共有  $m$  页，系统分配给它的主存块为  $n$  块，这里  $m > n$ 。
- 如果作业  $J_i$  执行过程中总的内存访问次数为  $A$ ，成功访问的次数为  $S$ ，不成功的访问次数为  $F$  (产生缺页中断的次数)，则：
  - $A = S + F$
  - 缺页率：  $f = F/A$



## 七、内存扩充 (2) 虚拟存储

### Performance of Demand Paging

- **Page Fault Rate  $0 \leq p \leq 1.0$**  (缺页率  $0 \leq p \leq 1.0$ )
  - if  $p = 0$  no page faults (如果  $p = 0$  , 没有缺页)
  - if  $p = 1$ , every reference is a fault (每次访问都缺页)
- **Effective Access Time (EAT)** (有效存取时间)

$$\begin{aligned} \text{EAT} = & (1 - p) \times \text{memory access} \\ & + p \times (\text{page fault overhead} \\ & + [\text{swap page out}] \\ & + [\text{swap page in}] \\ & + [\text{restart overhead}]) \end{aligned}$$

## 七、内存扩充 (2) 虚拟存储

### Page-Replacement Algorithms

- 在进程运行过程中，如果发生缺页，而内存中又无空闲块时，怎么办？
  - 将内存中的某一页换到磁盘的对换区
- 将哪个页面调出？
  - 根据页面置换算法来确定

## 七、内存扩充 (2) 虚拟存储

### Page-Replacement Algorithms

- Goal: Want lowest page-fault rate
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string (通过运行一个内存访问的特殊序列 (访问序列), 计算这个序列的缺页次数)
- 从理论上讲, 应将那些以后不再被访问的页面换出, 或把那些在较长时间内不会再被访问的页面换出。
- 存在多种置换算法, 都是试图更接近理论上的目标

# 七、内存扩充 (2) 虚拟存储

## Page-Replacement Algorithms

### 先进先出页面淘汰算法(FIFO)

- ❖ 思想：选择最早调入内存的页面淘汰。
- ❖ 根据：近期调入的页面被再次访问的概率要大于早期调入的页面。
- ❖ 举例：

设进程有6个虚页：0-5，页面访问顺序为P=4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5，可用主存页面大小(容量)M=3，采用FIFO进行页面淘汰。

| 时刻 | 1              | 2                   | 3                        | 4                        | 5                        | 6                        | 7                        | 8           | 9           | 10                       | 11                       | 12          |
|----|----------------|---------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|-------------|-------------|--------------------------|--------------------------|-------------|
| P  | 4              | 3                   | 2                        | 1                        | 4                        | 3                        | 5                        | 4           | 3           | 2                        | 1                        | 5           |
| M  | 4 <sup>+</sup> | 3 <sup>+</sup><br>4 | 2 <sup>+</sup><br>3<br>④ | 1 <sup>+</sup><br>2<br>③ | 4 <sup>+</sup><br>1<br>② | 3 <sup>+</sup><br>4<br>① | 5 <sup>+</sup><br>3<br>4 | 5<br>3<br>4 | 5<br>3<br>④ | 2 <sup>+</sup><br>5<br>③ | 1 <sup>+</sup><br>2<br>5 | 1<br>2<br>5 |
| F  | +              | +                   | +                        | +                        | +                        | +                        | +                        |             |             | +                        | +                        |             |

缺页中断次数F=9，而缺页率 $f=9/12=75\%$

问题：如果要减少缺页率，该怎么办？

# 七、内存扩充 (2) 虚拟存储

## Page-Replacement Algorithms

举例：

### 先进先出 (FIFO) 置换算法

设进程有6个虚页：0-5，页面访问顺序为P=4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5，可用主存页面大小(容量)M=3，采用FIFO进行页面淘汰。

主存容量M=4

| 时刻 | 1              | 2                   | 3                        | 4                        | 5                        | 6           | 7                        | 8                        | 9                        | 10                       | 11                       | 12                       |
|----|----------------|---------------------|--------------------------|--------------------------|--------------------------|-------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|
| P  | 4              | 3                   | 2                        | 1                        | 4                        | 3           | 5                        | 4                        | 3                        | 2                        | 1                        | 5                        |
| M  | 4 <sup>+</sup> | 3 <sup>+</sup><br>4 | 2 <sup>+</sup><br>3<br>4 | 1 <sup>+</sup><br>2<br>3 | 1 <sup>+</sup><br>2<br>3 | 1<br>2<br>3 | 5 <sup>+</sup><br>1<br>2 | 4 <sup>+</sup><br>5<br>1 | 3 <sup>+</sup><br>4<br>5 | 2 <sup>+</sup><br>3<br>4 | 1 <sup>+</sup><br>2<br>3 | 5 <sup>+</sup><br>1<br>2 |
|    |                |                     |                          | 4                        | 4                        | ④           | ③                        | ②                        | ①                        | ⑤                        | ④                        | 3                        |
| F  | +              | +                   | +                        | +                        |                          |             | +                        | +                        | +                        | +                        | +                        | +                        |

缺页中断次数F=10，而缺页率f=10/12=83%

**Belady现象：**可用页面数增大，缺页率反而升高的现象。  
为什么？  
如何解决？

## 七、内存扩充 (2) 虚拟存储

### Page-Replacement Algorithms

#### 最近最少使用页面淘汰算法(LRU, Least Recently Used)

- ❖ 思想：每次选择内存中离当前时刻最久未使用过的页面淘汰。
- ❖ 根据：程序执行的局部性原理。
- ❖ 举例

设进程有6个虚页：0-5，页面访问顺序为P=4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5，可用主存页面大小(容量)M=3，采用LRU算法进行页面淘汰。

| 时刻 | 1              | 2              | 3              | 4              | 5              | 6              | 7              | 8 | 9 | 10             | 11             | 12             |
|----|----------------|----------------|----------------|----------------|----------------|----------------|----------------|---|---|----------------|----------------|----------------|
| P  | 4              | 3              | 2              | 1              | 4              | 3              | 5              | 4 | 3 | 2              | 1              | 5              |
| M  | 4 <sup>+</sup> | 3 <sup>+</sup> | 2 <sup>+</sup> | 1 <sup>+</sup> | 4 <sup>+</sup> | 3 <sup>+</sup> | 5 <sup>+</sup> | 4 | 3 | 2 <sup>+</sup> | 1 <sup>+</sup> | 5 <sup>+</sup> |
|    |                | 4              | 3              | 2              | 1              | 4              | 3              | 5 | 4 | 3              | 2              | 1              |
|    |                |                | ④              | ③              | ②              | ①              | 4              | 3 | ⑤ | ④              | ③              | 2              |
| F  | +              | +              | +              | +              | +              | +              | +              |   |   | +              | +              | +              |

缺页中断次数F=10，缺页率 $f=10 / 12=83\%$

## 七、内存扩充 (2) 虚拟存储

最近最少

- ❖ 思想:
- ❖ 根据:
- ❖ 举例

| 时刻 | 1              | 2                   | 3                        | 4                        | 5                        | 6                        | 7                        | 8           | 9           | 10                       | 11                       | 12          |
|----|----------------|---------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|-------------|-------------|--------------------------|--------------------------|-------------|
| P  | 4              | 3                   | 2                        | 1                        | 4                        | 3                        | 5                        | 4           | 3           | 2                        | 1                        | 5           |
| M  | 4 <sup>+</sup> | 3 <sup>+</sup><br>4 | 2 <sup>+</sup><br>3<br>④ | 1 <sup>+</sup><br>2<br>③ | 4 <sup>+</sup><br>1<br>② | 3 <sup>+</sup><br>4<br>① | 5 <sup>+</sup><br>3<br>4 | 5<br>3<br>4 | 5<br>3<br>④ | 2 <sup>+</sup><br>5<br>③ | 1 <sup>+</sup><br>2<br>5 | 1<br>2<br>5 |
| F  | +              | +                   | +                        | +                        | +                        | +                        | +                        |             |             | +                        | +                        |             |

设进程有6个虚页：0-5，页面访问顺序为P=4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5，可用主存页面大小(容量)M=3，采用LRU算法进行页面淘汰。

| 时刻 | 1              | 2                   | 3                        | 4                        | 5                        | 6                        | 7                        | 8           | 9           | 10                       | 11                       | 12                       |
|----|----------------|---------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|-------------|-------------|--------------------------|--------------------------|--------------------------|
| P  | 4              | 3                   | 2                        | 1                        | 4                        | 3                        | 5                        | 4           | 3           | 2                        | 1                        | 5                        |
| M  | 4 <sup>+</sup> | 3 <sup>+</sup><br>4 | 2 <sup>+</sup><br>3<br>④ | 1 <sup>+</sup><br>2<br>③ | 4 <sup>+</sup><br>1<br>② | 3 <sup>+</sup><br>4<br>① | 5 <sup>+</sup><br>3<br>4 | 4<br>5<br>3 | 3<br>4<br>⑤ | 2 <sup>+</sup><br>3<br>④ | 1 <sup>+</sup><br>2<br>③ | 5 <sup>+</sup><br>1<br>2 |
| F  | +              | +                   | +                        | +                        | +                        | +                        | +                        |             |             | +                        | +                        | +                        |

缺页中断次数F=10，缺页率 $f=10 / 12=83\%$

# 七、内存扩充 (2) 虚拟存储

## Page-Replacement Algorithms

举例

最近最少使用LRU算法

设进程有6个虚页：0-5，页面访问顺序为P=4, 3, 2, 1, 4, 3, 5, 4, 3, 2, 1, 5，可用主存页面大小(容量)M=3，采用LRU算法进行页面淘汰。

主存容量M=4

| 时刻 | 1              | 2                   | 3                        | 4                             | 5                | 6                | 7                             | 8                | 9                | 10                            | 11                            | 12                            |
|----|----------------|---------------------|--------------------------|-------------------------------|------------------|------------------|-------------------------------|------------------|------------------|-------------------------------|-------------------------------|-------------------------------|
| P  | 4              | 3                   | 2                        | 1                             | 4                | 3                | 5                             | 4                | 3                | 2                             | 1                             | 5                             |
| M  | 4 <sup>+</sup> | 3 <sup>+</sup><br>4 | 2 <sup>+</sup><br>3<br>4 | 1 <sup>+</sup><br>2<br>3<br>4 | 4<br>1<br>2<br>3 | 3<br>4<br>1<br>② | 5 <sup>+</sup><br>3<br>4<br>1 | 4<br>5<br>3<br>1 | 3<br>4<br>5<br>① | 2 <sup>+</sup><br>3<br>4<br>⑤ | 1 <sup>+</sup><br>2<br>3<br>④ | 5 <sup>+</sup><br>1<br>2<br>3 |
| F  | +              | +                   | +                        | +                             |                  |                  | +                             |                  |                  | +                             | +                             | +                             |

缺页中断次数F=8，缺页率 $f=8/12=67\%$

结论：LRU算法不会出现Belady现象。  
为什么？



## 七、内存扩充 (2) 虚拟存储

### Least Recently Used (LRU) Algorithm

- Counter implementation (计数器实现)

- Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter (每个页表项都有一个**计数器**；每次通过该条目访问页面时，将**时钟值**复制到计数器中)
- When a page needs to be changed, look at the counters to determine which are to change (当需要更改页面时，**查看计数器**以确定哪些需要更改) (**最久未用**)

- 优点：逻辑直观，直接用“时间戳”标记页的使用新旧
- 缺点：每次访问页都要写内存更新计数器（性能开销）；置换时需遍历页表找最小值（耗时）

## 七、内存扩充 (2) 虚拟存储

### Least Recently Used (LRU) Algorithm

- **Stack implementation** – keep a stack of page numbers in a double link form (**栈实现**—用一个**双向链表**实现一个记录页号的栈) :
  - Page referenced (被访问的页移到栈顶)
  - **栈底**的页是**最近最少被访问的** (**最久未用页**)
  - No search for replacement (不用为置换进行查找)
  - 每次把一个页号从栈中移动到栈顶, 需修改几个指针
- 优点: 置换时无需遍历 (栈底就是目标) ; 访问页只需调整链表指针 (比计数器写内存快)
- 缺点: 依赖链表操作 (实现稍复杂) ; 需维护双向链表的指针修改

## 七、内存扩充（2） 虚拟存储

### 最佳置换（Optimal Page Replacement, OPT）算法

- ◆ 最佳置换算法选择淘汰的页面是**以后永不使用的页面**，或是在**最长时间****内不再被访问的页面**，以便保证获得最低的缺页率。
- ◆ 然而，目前无法真正预测进程在内存的若干页面中哪个是未来最长时间  
内不再被访问的，因此该算法实际无法实现，但可利用该算法去评价其  
他算法。

# 七、内存扩充（2） 虚拟存储

## 最佳置换（Optimal Page Replacement, OPT）算法

【举例】假定系统为某进程分配了三个物理块，访问序列如下：

7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

进程运行时，先将 7,0,1 三个页面依次装入内存。当**进程要访问页面 2 时**，产生**缺页中断**，根据最佳置换算法，选择**将第 18 次访问才需调入的页面 7 淘汰**。然后，访问页面 0 时，因为它已在内存中，所以不必产生缺页中断。访问页面 3 时，又会根据最佳置换算法将页面 1 淘汰…… 以此类推。

| 访问页面  | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 | 1 | 2 | 0 | 1 | 7 | 0 | 1 |
|-------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 物理块 1 | 7 | 7 | 7 | 2 |   | 2 |   | 2 |   |   | 2 |   |   | 2 |   |   |   | 7 |   |   |
| 物理块 2 |   | 0 | 0 | 0 |   | 0 |   | 4 |   |   | 0 |   |   | 0 |   |   |   | 0 |   |   |
| 物理块 3 |   |   | 1 | 1 |   | 3 |   | 3 |   |   | 3 |   |   | 1 |   |   |   | 1 |   |   |
| 缺页否   | √ | √ | √ | √ |   | √ |   | √ |   |   | √ |   |   | √ |   |   |   | √ |   |   |

缺页中断的次数为 9，页面置换的次数为 6

# 七、内存扩充 (2) 虚拟存储

## 时钟 (CLOCK) 置换算法

### ① 简单的 CLOCK 置换算法，也称最近未用 (NRU) 算法

- 为页面设访问位，**首次装入或访问时置 1**。
- 内存页面成循环队列，配替换指针，**替换时指向下一页**。
- **选淘汰页时**，查访问位：**为 0 则换出**；**为 1 则置 0**，**给二次驻留机会**，继续检查下页。
- 若到队尾访问位仍为 1，返回队首循环检查。

# 七、内存扩充（2） 虚拟存储

【举例】假设页面访问顺序为 7,0,1,2,0,3,0,4,2,3,0,3,2,1,3,2，采用简单 CLOCK 置换算法，分配 4 个页帧，每个页对应的结构为（**页面号**，**访问位**）。

| 访问顺序 | 7 |   | 0 |   | 1 |   | 2 |   | 0 |   | 3 |   | 0 |   | 4 |   | 2 |   | 3 |   | 0 |   | 3 |   | 2 |   | 1 |   | 3 |   | 2 |   |
|------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 帧 1  | 7 | 1 | 7 | 1 | 7 | 1 | 7 | 1 | 7 | 1 | 3 | 1 | 3 | 1 | 3 | 1 | 3 | 1 | 3 | 1 | 3 | 1 | 3 | 1 | 3 | 1 | 3 | 0 | 3 | 1 | 3 | 0 |
| 帧 2  |   |   | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 2 | 1 |   |
| 帧 3  |   |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 0 | 4 | 1 | 4 | 1 | 4 | 1 | 4 | 1 | 4 | 1 | 4 | 1 | 4 | 0 | 4 | 0 | 4 | 0 |
| 帧 4  |   |   |   |   |   |   | 2 | 1 | 2 | 1 | 2 | 0 | 2 | 0 | 2 | 0 | 2 | 1 | 2 | 1 | 2 | 1 | 2 | 1 | 2 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 缺页否  | √ |   | √ |   | √ |   | √ |   |   |   | √ |   |   |   | √ |   |   |   |   |   |   |   |   |   |   |   | √ |   |   |   | √ |   |

- 首次访问 7、0、1、2，缺页中断，调入主存，访问位置 1。访问0，已存在，访问位为1。
- 访问 3（第 5 次缺页）：替换指针从帧 1 开始扫描，此时所有帧访问位为 1，扫描一周置访问位为 0，回帧 1，替换帧 1 的页，访问位置 1（替换指针此时在帧2）。
- 访问 0：已存在，访问位置 1；访问 4（第 6 次缺页）：指针在帧 2，帧 2 访问位置 0，继续扫到帧 3（访问位 0），替换帧 3 的页（替换指针此时在帧4）。
- 访问 2、3、0、3、2：已存在，访问位置 1；访问 1（缺页）：指针到帧 4，因所有帧访问位为 1，扫描一周置访问位为 0，回帧 4，替换帧 4 的页（替换指针此时在帧1）。
- 访问 3：已存在，访问位置 1；访问 2（缺页）：指针到帧 1，帧 1 访问位置 0，继续扫到帧 2（访问位 0），替换帧 2 的页。

## 七、内存扩充 (2) 虚拟存储

### 时钟 (CLOCK) 置换算法

#### ② 改进的 CLOCK 置换算法

将一个页面换出时，若该页已被修改过，则要将该页写回磁盘，若该页未被修改过，则不必将它写回磁盘。可见，对于**修改过的页面**，将其**替换代价更大**。

在改进型 CLOCK 算法中，除考虑页面使用情况外，还增加了**置换代价——修改位**。在选择页面换出时，优先考虑**既未使用过又未修改过的页面**。由访问位 A 和修改位 M 可以组合成下面四种类型的页面：

- **1 类**  $A=0, M=0$ ：最近未被访问，且未被修改，是最佳的淘汰页。
- **2 类**  $A=0, M=1$ ：最近未被访问，但已被修改，是次佳的淘汰页。
- **3 类**  $A=1, M=0$ ：最近已被访问，但未被修改，可能再被访问。
- **4 类**  $A=1, M=1$ ：最近已被访问，且已被修改，可能再被访问。

内存中的每页必定都是这四类页面之一。在进行页面置换时，可采用与简单 CLOCK 算法类似的算法，差别在于该算法要同时检查访问位和修改位。

# 七、内存扩充 (2) 虚拟存储

## 时钟 (CLOCK) 置换算法

### ② 改进的 CLOCK 置换算法

算法执行过程如下：

- 第一轮扫描：寻找 $A=0, M=0$ 的页面，找到即置换，扫描中**不修改访问位A**。
- 第二轮扫描：若失败，寻找 $A=0, M=1$ 的页面，找到即置换；此轮扫描中**将经过页面的A置0**。
- 重置重试：若仍失败，指针归位，**所有A清零**，重新执行上述步骤。

算法优势

- 优先置换未修改的页面，减少磁盘I/O次数。
- 综合考虑访问与修改状态，置换策略更精细。

**核心原则：**保留已访问页面，优先淘汰“未使用”或“未修改”的页面，平衡系统开销与置换效率。



## 七、内存扩充（2） 虚拟存储

### 页框回收——页面缓冲算法

在页式虚拟存储系统中，页面换入 / 换出的开销会对系统性能产生重大影响。**影响页面换入 / 换出效率的因素主要包括：**

(1) **页面置换算法**。好的页面置换算法可使进程在运行过程中具有较低的缺页率，从而减少页面换入 / 换出的开销。

(2) **将已修改页面写回磁盘的频率**。对于已被修改的页面，将其换出时，应写回磁盘。若每当有一个页面要被换出时就将它写回磁盘，则会导致很大的磁盘 I/O 开销。

(3) **将磁盘内容读入内存的频率**。若进程的每次访问都要将磁盘内容读入内存，则会导致非常高的磁盘 I/O 频率，进而增加页面换入的开销。

## 七、内存扩充（2） 虚拟存储

### 页框回收——页面缓冲算法

在原页面置换算法基础上，增设**已修改页面链表**，攒够一定数量已修改且需换出的页面后，再一并换出到磁盘，减少页面换出开销。

#### ■ 主要特点：

显著降低页面换入 / 换出频率，大幅减少磁盘 I/O 开销；

因换入 / 换出开销减小，采用简单置换策略（如 FIFO）时，无需特殊硬件，实现简单。

#### ■ 两个关键链表：

- **空闲页面链表**：进程读入页面时，从链首取页框装入；未修改页面换出时，不换至磁盘，将页框挂到链尾（页框含数据，后续进程可直接取，减少换入开销）。
- **修改页面链表**：已修改页面换出时，先将页框挂到链表末尾，不立即换至磁盘，降低已修改页面写回磁盘及磁盘内容读入内存的频率。

## 七、内存扩充（2） 虚拟存储

### Page-Replacement Algorithms

- 置换算法的好坏将直接影响系统的性能，不适当的算法可能会导致进程发生“抖动/颠簸”（Thrashing）
- 抖动：刚被换出的页很快又被访问，需重新调入，导致系统频繁地交换页面，以致大部分CPU时间花费在完成页面置换的工作上

# 七、内存扩充 (2) 虚拟存储

## 虚拟页式存储管理方案性能分析

### ❖ 颠簸/抖动(thrashing)现象

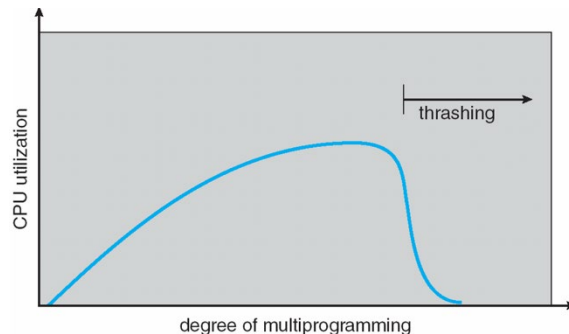
- 页面在内存与外存之间频繁调度，以至于调度页面所需时间比进程实际运行的时间还多，此时系统效率急剧下降，甚至导致系统崩溃。这种现象称为**颠簸或抖动**。

- 原因：

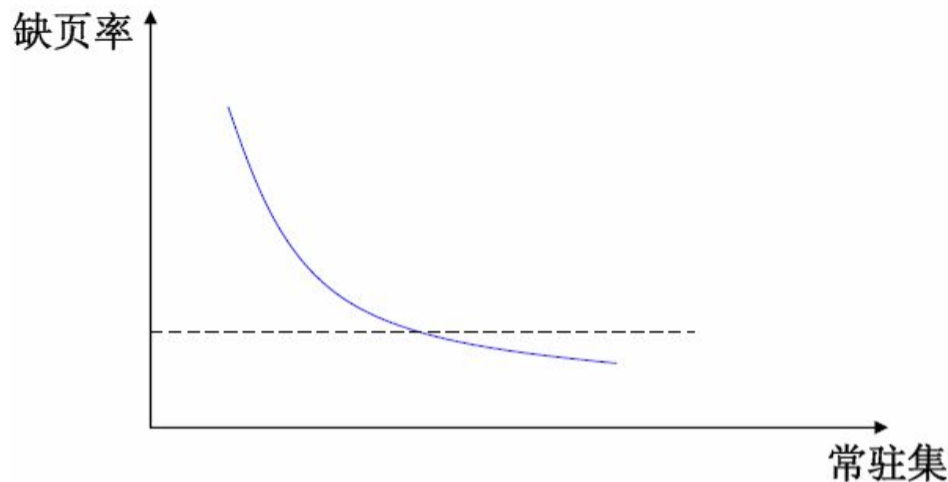
- ① 页面淘汰算法不合理；
- ② 分配给进程的物理页面数太少。

### ❖ 常驻集(resident set)

- **常驻集**指虚拟页式管理中给进程分配的物理页面数目。
- 常驻集和缺页率的关系：
  - ① 常驻集越小，内存中能够容纳的并发进程数目越多，但就单个进程来说，缺页中断率会上升，调页开销增大。
  - ② 常驻集达到一定数目后，再增加页面，缺页率不会明显下降。



## 七、内存扩充 (2) 虚拟存储



### ❖ 常驻集大小的确定

- **固定分配法**：常驻集大小固定。常用方法：平均分配法；根据程序大小按比例分配法；按优先权分配。
- **可变分配**：常驻集大小可变，可按照缺页率动态调整。性能较好，但增加了算法开销。

# 七、内存扩充 (2) 虚拟存储

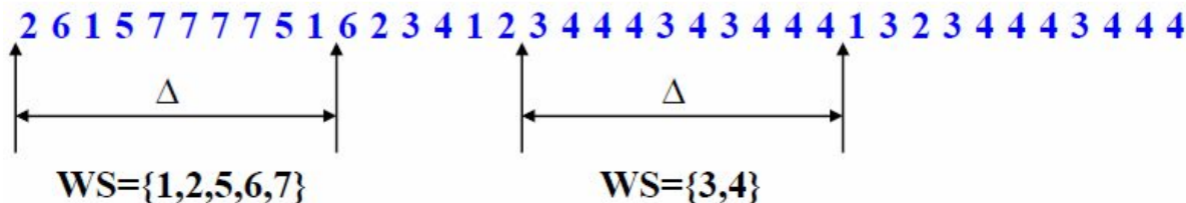
## 工作集(working-set)

### ❖ 引入目的

- 依据进程在过去的一段时间内访问的页面来调整常驻集大小。

### ❖ 概念

- 根据程序的局部性原理，一般情况下，进程在一段时间内总是集中访问一些页面，这些页面称为**活跃页面**。如果分配给一个进程的物理页面数太少了，使该进程所需的活跃页面不能全部装入内存，则进程在运行过程中将频繁发生中断。如果能为进程提供与活跃页面数相等(或大于)的物理页面数，则可减少缺页中断次数。对于给定的访问序列选取定长的区间( $\Delta$ )，称为**工作集窗口**，落在工作集窗口中的页面集合称为工作集。

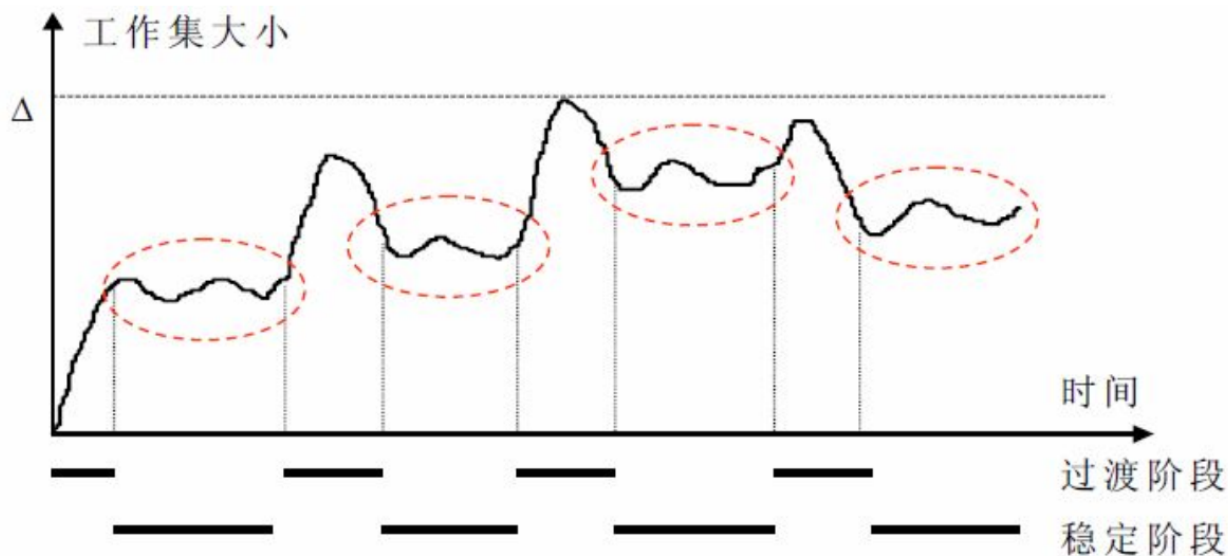


## 七、内存扩充 (2) 虚拟存储

- **工作集的具体实现：**
  - OS跟踪每个进程的工作集，并为其分配大于其工作集的物理块数。
  - 如果还有空闲物理块，则可启动另外的进程。
  - 如果所有进程的工作集之和超过了可用物理块的总数，则OS会选择暂停一个进程，该进程被换出，所释放的物理块可分配给其他进程。
- **工作集模型**本身不是一个直接的页面置换算法（如FIFO、LRU），而是一个**指导性原则**。
- 基于工作集模型的经典置换算法是 **WSClock 算法**。它结合了工作集的想法和时钟算法的简单性——使用一个**循环链表**来管理页面：当发生缺页中断时，算法扫描链表：
  - 如果引用位为1，说明该页在工作集中，清除引用位并跳过。
  - 如果引用位为0，则检查该页的“上次使用时间”。若此时间超出了工作集窗口  $\Delta$ ，就认为该页已不在工作集中，可以置换。否则，保留。

## 七、内存扩充 (2) 虚拟存储

### ➤ 工作集大小的变化规律

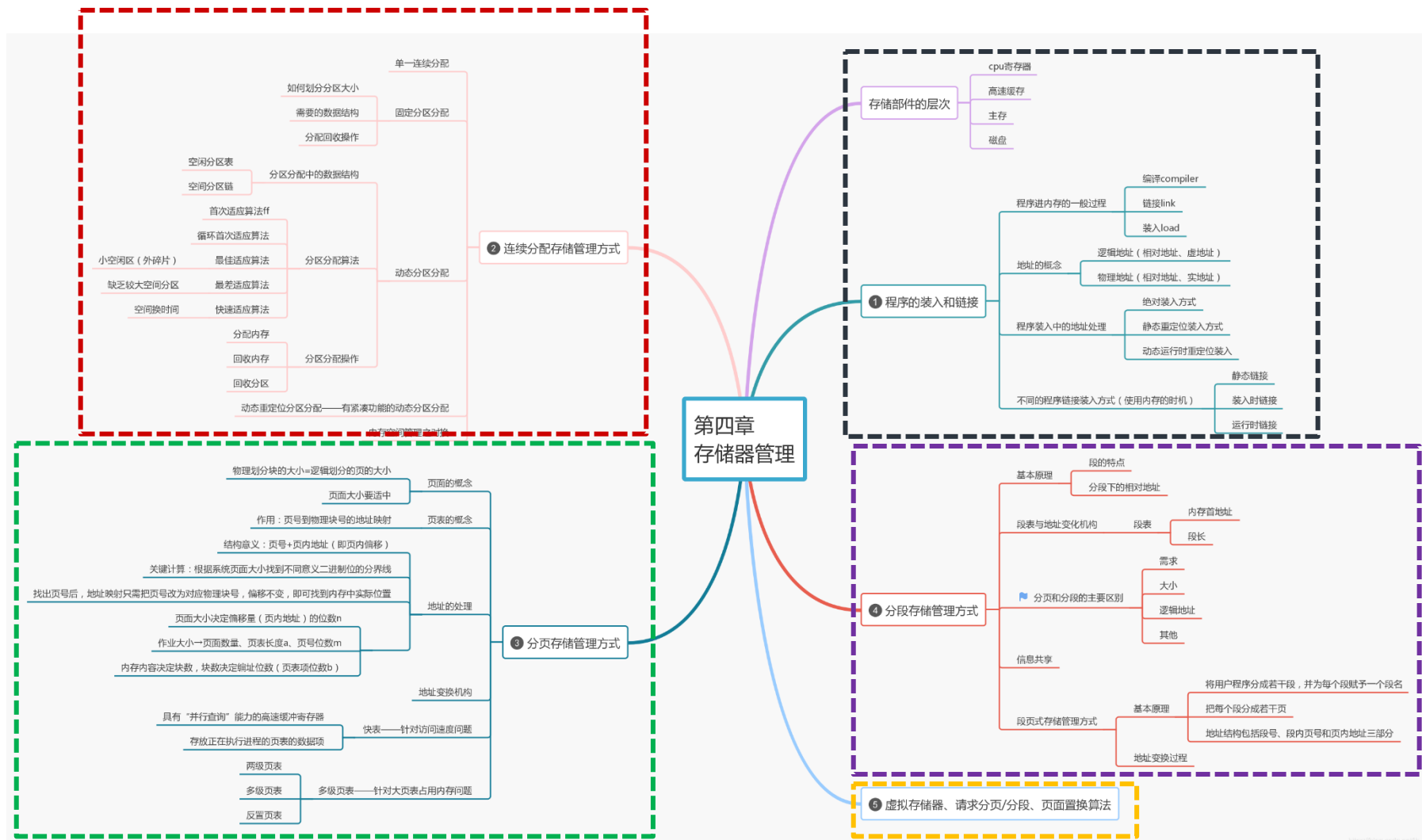


### ➤ 利用工作集进行常驻集调整的策略

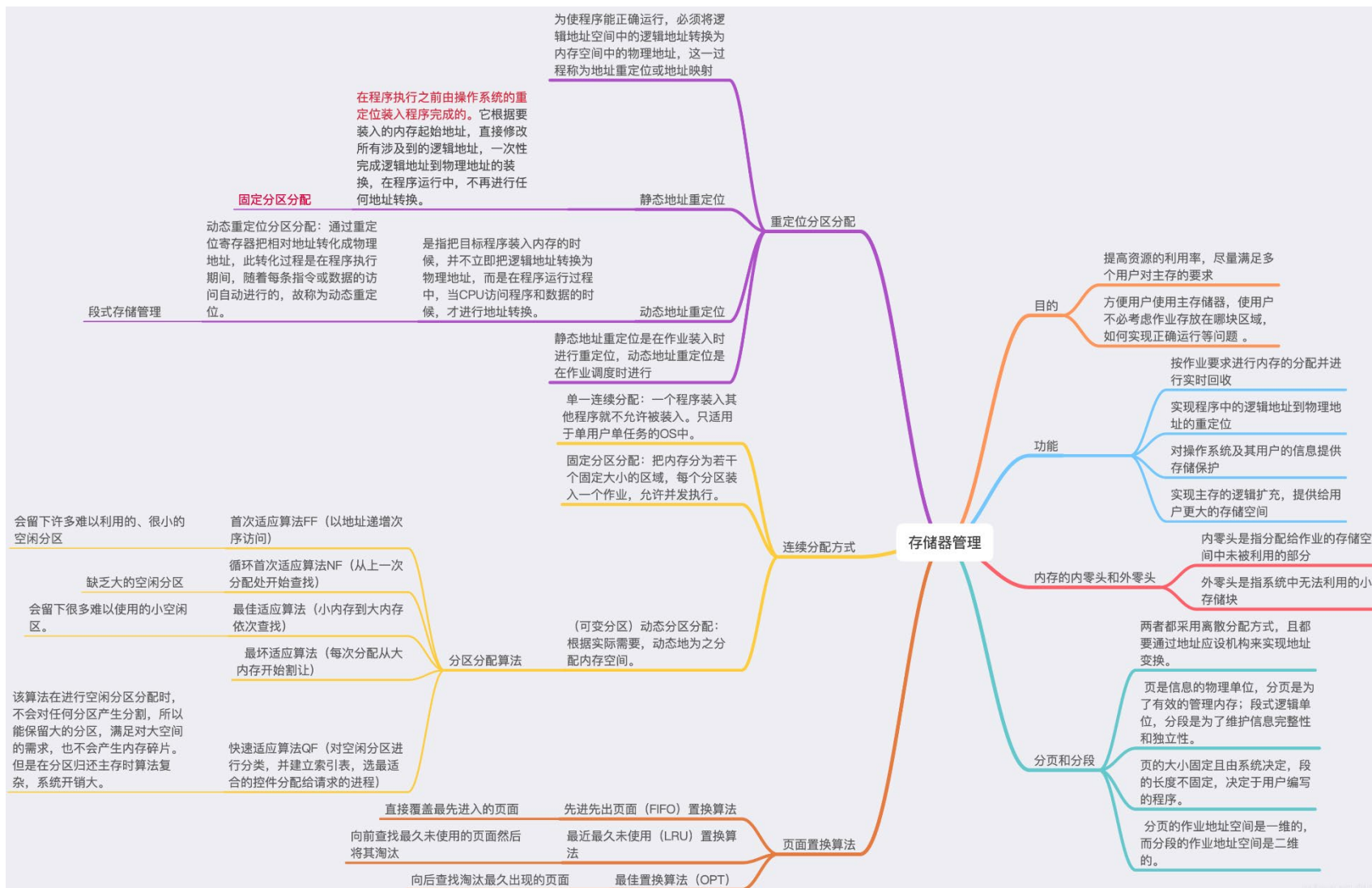
- ① 记录一个进程的工作集变化;
- ② 定期从常驻集中删除不在工作集中的页面;
- ③ 总是让常驻集包含工作集。



# 小结



# 小结



# 作业（选做）

论述：程序的运行是否需要内存参与？是如何实现的？

论述：页式、段式存储管理对应的数据结构和硬件。

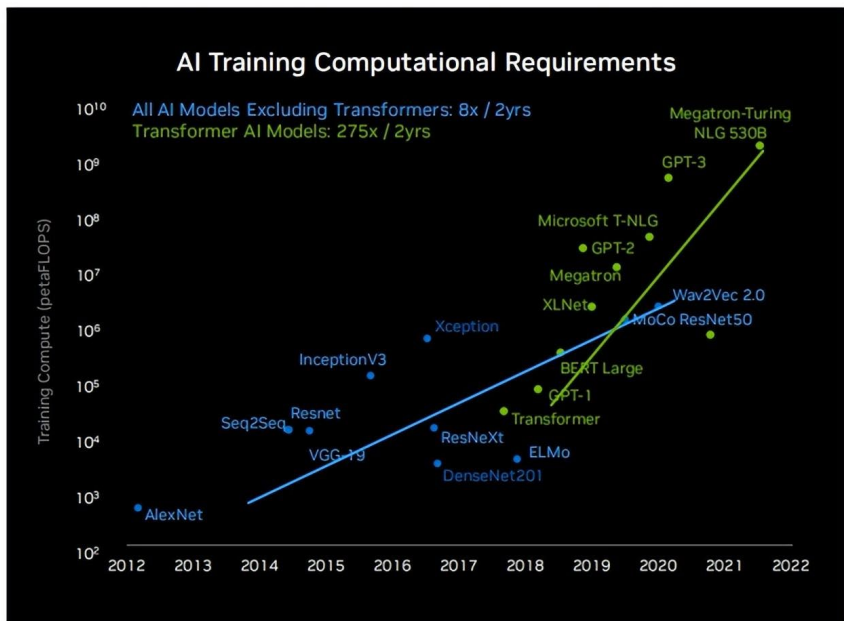
论述：参照课堂学习的页式和请求页式存储管理，自学段氏与请求段氏存储管理。

**思考：**AI时代下，新的存储和计算形态

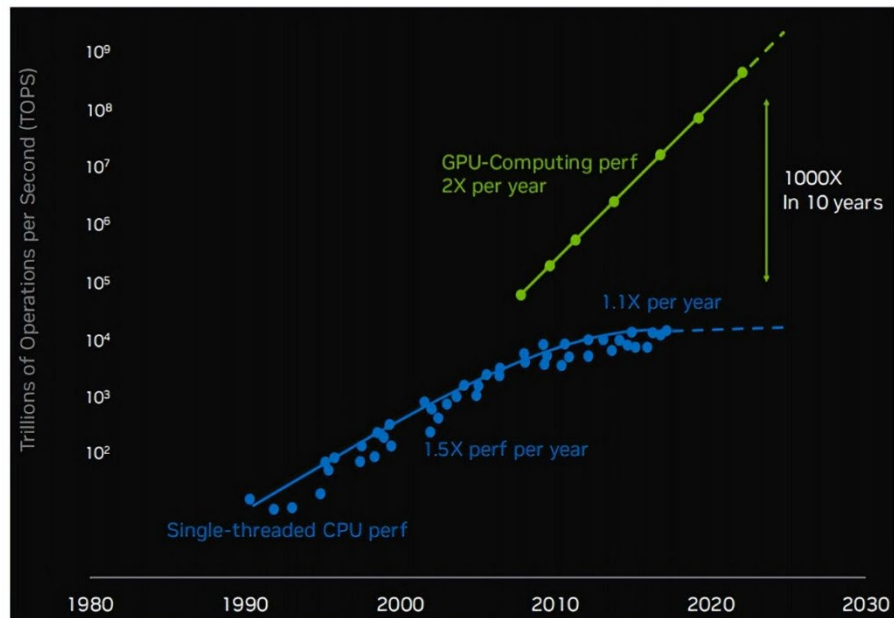
# 关于“存储”的讨论与思考

## ❖ 大模型的算力基座

### 大模型训练算力需求2年翻275倍



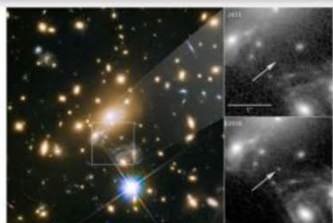
### GPU计算性能未来10年再翻1000倍



## 关于“存储”的讨论与思考

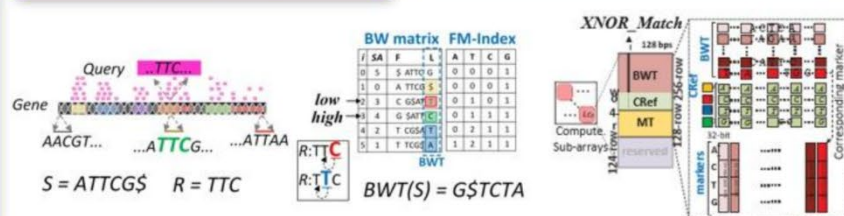
## ❖ 大模型的算力基座

## 大数据检索/分析

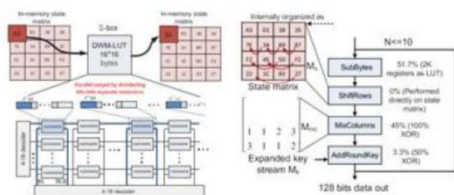


| (a) | Dist. | Size   | Year | (b)    | A | B | C | D | E | F | G | H |
|-----|-------|--------|------|--------|---|---|---|---|---|---|---|---|
| A   | 55    | Large  | 2016 | Fear   | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
| B   | 23    | Medium | 2014 | Near   | 0 | 1 | 0 | 0 | 1 | 1 | 1 | 1 |
| C   | 43    | Small  | 2015 | Large  | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| D   | 60    | Medium | 2016 | Medium | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 0 |
| E   | 25    | Medium | 2000 | Small  | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
| F   | 34    | Medium | 2001 | New    | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 0 |
| G   | 18    | Small  | 2012 | Old    | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 |
| H   | 30    | Small  | 2011 | OR     | 1 | 0 | 1 | 1 | 0 | 0 | 0 | 0 |
|     |       |        |      | AND    | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |

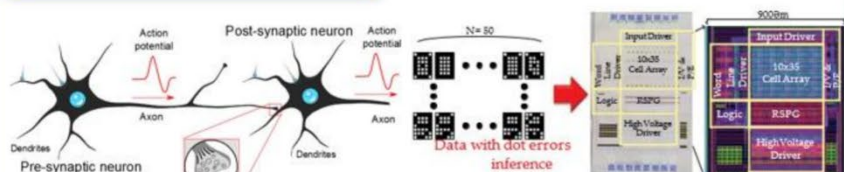
## 蛋白质/基因分析



## 可信计算/数据加密



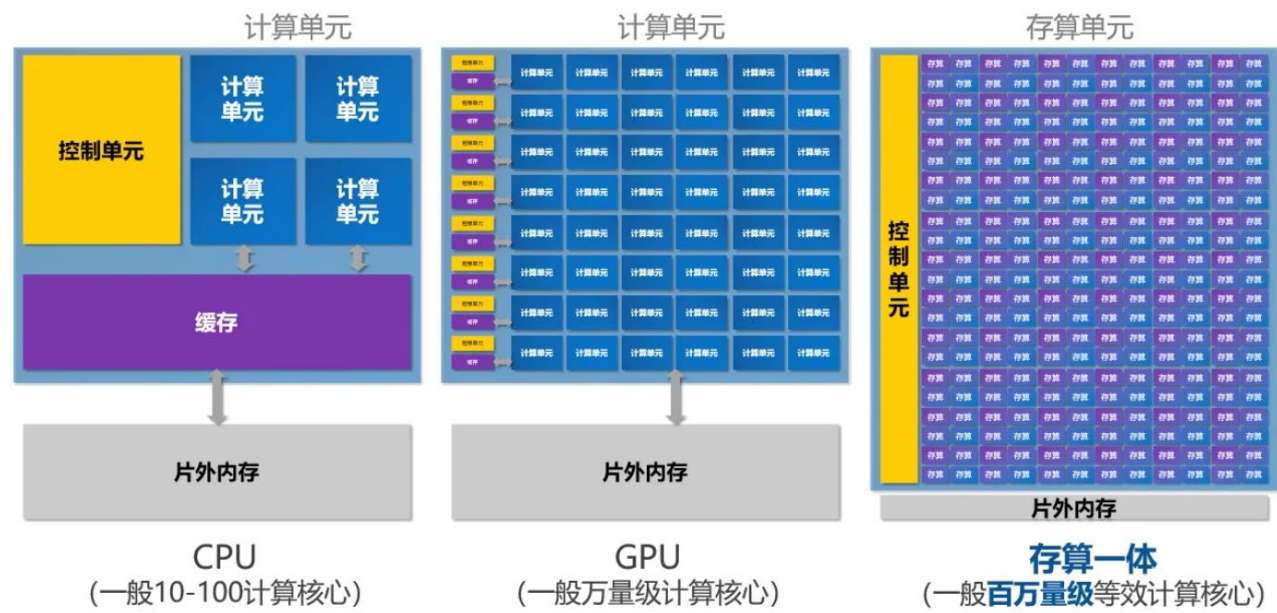
## 类脑计算





# 关于“存储”的讨论与思考

## ❖ 大模型的算力基座



| 器件基本特性 |      |      |      |      |     |      |     |
|--------|------|------|------|------|-----|------|-----|
| 特性     | DRAM | SRAM | RRAM | MRAM | PCM | NAND | HDD |
| 读功耗    | 低    | 低    | 低    | 中    | 低   | 高    | 高   |
| 写功耗    | 低    | 高    | 低    | 中    | 高   | 高    | 高   |
| 读写以外功耗 | 刷新功耗 | 漏功耗  | 无    | 无    | 无   | 无    | 无   |
| 非易失性   | 否    | 否    | 是    | 是    | 是   | 是    | 是   |

| 工艺、制程、环境及适用性 |                      |                             |                        |
|--------------|----------------------|-----------------------------|------------------------|
| 存储器类型        | 优势                   | 不足                          | 适用场景                   |
| SRAM         | 工艺成熟，适合IP化           | 对PVT变化敏感，对信噪比敏感，存储密度低，静态功耗高 | 小算力，端侧，不要求待机功耗         |
| DRAM         | 高存储密度，整合方案成熟         | 只能做近存计算，速度低，工艺迭代慢           | 适合现有冯氏架构向存算过渡          |
| Flash        | 高密度低成本，非易失，低漏电       | 对PVT变化敏感，精度不高，工艺迭代时间长       | 小算力，端侧，待机时间长的场景        |
| RRAM         | 算力大，能效比高，高密度，非易失，低漏电 | 有限写次数，差异化工艺良率               | 大算力，端侧/边缘/中心侧，待机时间长的场景 |

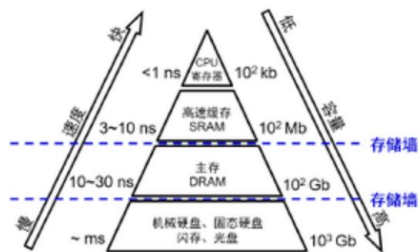
# 关于“存储”的讨论与思考

## ❖ 存储的方式、与计算的关系

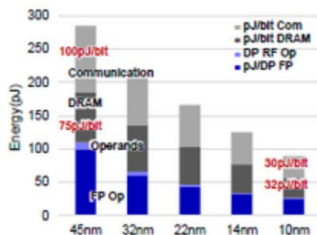
现有计算系统普遍采用存储和运算分离的架构，存在“存储墙”与“功耗墙”瓶颈，严重制约了系统算力和能效的提升。



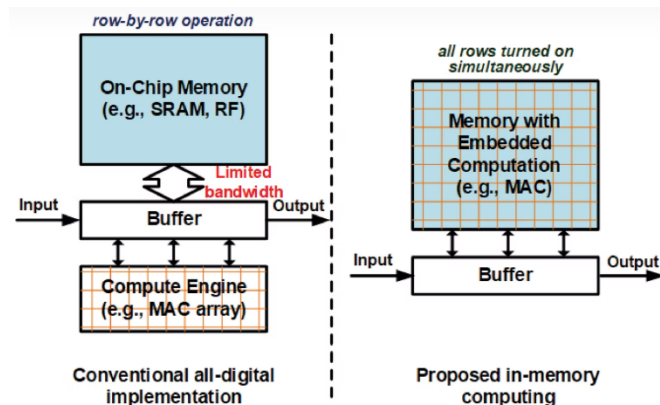
冯诺依曼架构



存储速度远低于计算速度



10nm工艺下，数据传输和访问功耗占比超过69%



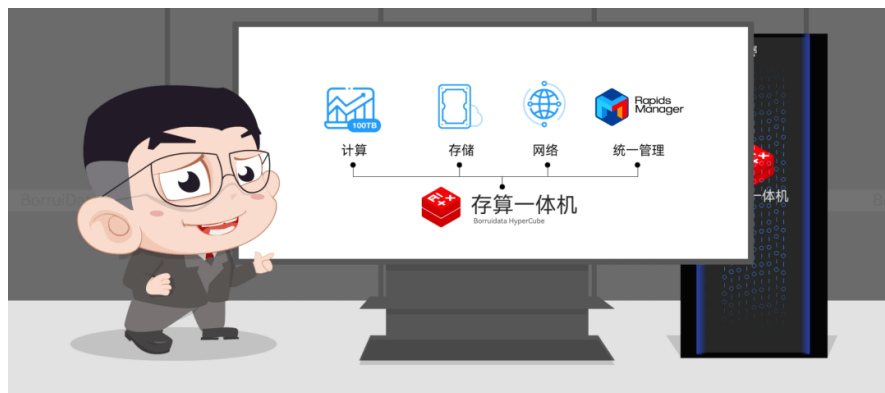
### 存算一体

- 优势：
  - 针对固定硬件配置做极致优化，单机性能较好；
- 劣势：
  - 使用专用服务器或硬件配置较固定，硬件的通用性和灵活性不足；
  - 集群水平扩展、垂直扩展不灵活，单独扩展计算或存储较困难；
  - 集群扩展后需要做数据重分布，期间影响甚至停止对外服务；

VS

### 存算解耦

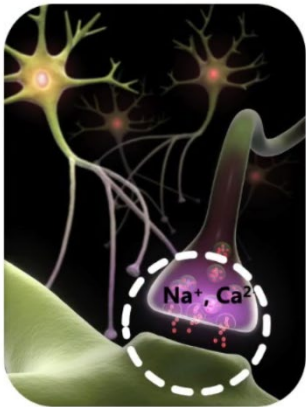
- 优势：
  - 使用通用型服务器，可配置范围广，硬件的通用性和灵活性高；
  - 集群水平扩展、垂直扩展灵活，可以按需单独扩展计算或存储；
  - 集群扩展后自动管理数据均匀分布，无需数据重分布操作，连续提供对外服务；
- 劣势：
  - 为保持硬件的通用性和灵活性，单机性能优化较一般。



# 关于“存储”的讨论与思考

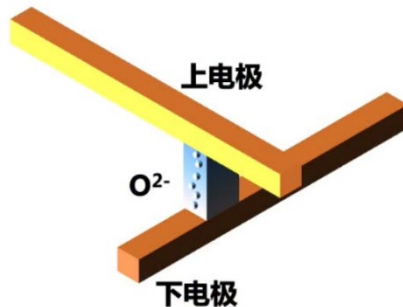
## ❖ 存储的方式、与计算的关系——忆阻器

生物突触

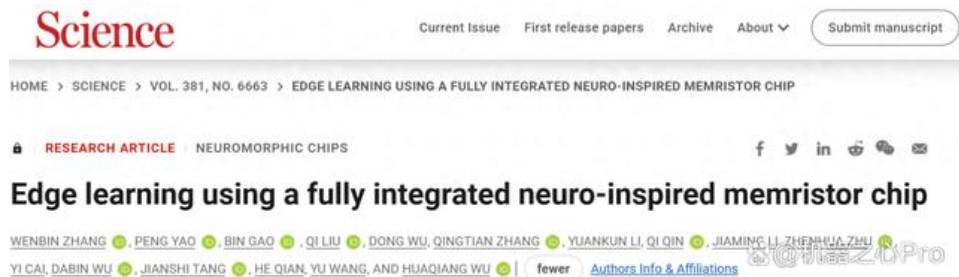


生物中  $\text{Na}^+$ ,  $\text{Ca}^{2+}$  离子的移动造成生物突触的强弱。

RRAM 电子突触



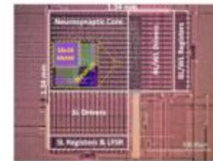
RRAM中氧离子的移动和分布导致器件阻值的变化。



松下公司  
宏电路  
多层感知机  
(VLSI 2018)



新竹清华大学和台积电  
宏电路  
卷积核计算  
(ISSCC 2019)



斯坦福与清华合作  
宏电路  
限制玻尔兹曼机  
(ISSCC 2020)



清华大学  
全系统集成的完整存算一体芯片  
多层感知机  
(ISSCC 2020)

相关论文地址:

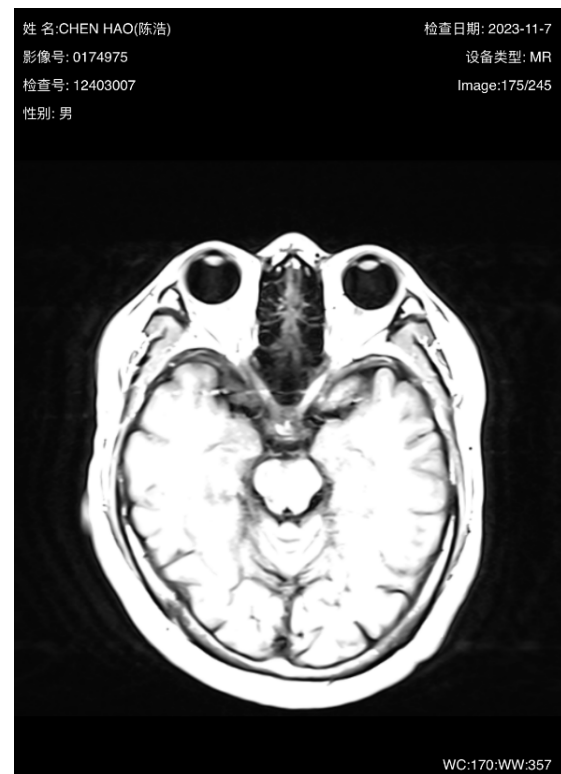
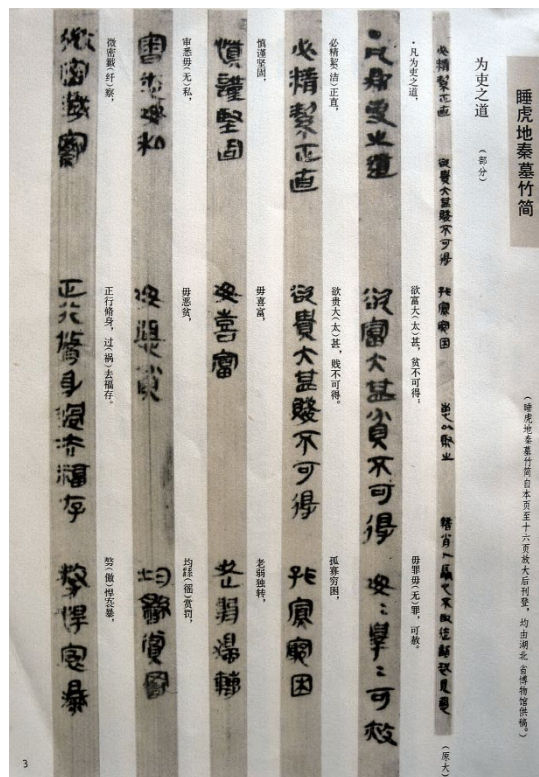
<https://www.science.org/doi/full/10.1126/science.ade3483>

<https://www.nature.com/articles/s41586-022-04992-8>



# 小结

## ❖ 存储与记忆



持久化、虚拟化

## ❖ 存储与记忆



## ❖ 存储与记忆

| 国内外关注度较高的模型上下文可接受长度表         |              |           | 光维智能 |
|------------------------------|--------------|-----------|------|
| 公司/机构/团队                     | 模型/产品名称      | 上下文Tokens |      |
| OpenAI                       | GPT-3.5      | 4K-16K    |      |
|                              | GPT-4        | 8K-32K    |      |
| Anthropic                    | Claude       | 100K      |      |
|                              | Claude2      | 100K      |      |
| Meta                         | LLaMA        | 2k        |      |
|                              | LLaMA2       | 4k        |      |
|                              | Llama 2 Long | 32K       |      |
| IDEAS NCBR 、Google DeepMind等 | Long LLaMA   | 256k      |      |
| Moonshot（月之暗面）               | Kimi Chat    | 400K      |      |
| 港中文贾佳亚团队、MIT                 | LongAlpaca   | 32K-100K  |      |

## 大模型能记忆的上下文长度



## ❖ 存储安全与攻防—— 内存溢出（Out of Memory Error）、内存泄露

```

350. at com.alibaba.dubbo.remoting.exchange.support.header.HeaderExchangeHandler.handleRequest(HeaderExchangeHandler.java:95) [dubbo-2.5.4-thzk-13.jar:]
351. at com.alibaba.dubbo.remoting.exchange.support.header.HeaderExchangeHandler.received(HeaderExchangeHandler.java:215) [dubbo-2.5.4-thzk-13.jar:]
352. at com.alibaba.dubbo.remoting.transport.DecodeHandler.received(DecodeHandler.java:58) [dubbo-2.5.4-thzk-13.jar:]
353. at com.alibaba.dubbo.remoting.transport.dispatcher.ChannelEventRunnable.run(ChannelEventRunnable.java:99) [dubbo-2.5.4-thzk-13.jar:]
354. at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1145) [rt.jar:1.7.0_75]
355. at java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:615) [rt.jar:1.7.0_75]
356. at java.lang.Thread.run(Thread.java:745) [rt.jar:1.7.0_75]
357. Caused by: java.lang.OutOfMemoryError: Java heap space
358. at java.util.Arrays.copyOf(Arrays.java:2367) [rt.jar:1.7.0_75]
359. at java.lang.AbstractStringBuilder.expandCapacity(AbstractStringBuilder.java:130) [rt.jar:1.7.0_75]
360. at java.lang.AbstractStringBuilder.ensureCapacityInternal(AbstractStringBuilder.java:114) [rt.jar:1.7.0_75]
361. at java.lang.AbstractStringBuilder.append(AbstractStringBuilder.java:415) [rt.jar:1.7.0_75]
362. at java.lang.StringBuilder.append(StringBuilder.java:132) [rt.jar:1.7.0_75]
363. at org.springframework.aop.aspectj.AbstractAspectJAdvice.invokeAdviceMethodWithBeforeAndAfter(AbstractAspectJAdvice.java:641) [spring-aop-3.2.13.RELEASE.jar:3.2.13.RELEASE]
364. at sun.reflect.NativeMethodAccessorImpl.invoke0(Native Method) [rt.jar:1.7.0_75]
365. at sun.reflect.NativeMethodAccessorImpl.invoke(NativeMethodAccessorImpl.java:57) [rt.jar:1.7.0_75]
366. at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43) [rt.jar:1.7.0_75]
367. at java.lang.reflect.Method.invoke(Method.java:606) [rt.jar:1.7.0_75]
368. at org.springframework.aop.support.AopUtils.invokeJoinpointUsingReflection(AopUtils.java:317) [spring-aop-3.2.13.RELEASE.jar:3.2.13.RELEASE]
369. at org.springframework.aop.framework.ReflectiveMethodInvocation.invokeJoinpoint(ReflectiveMethodInvocation.java:183) [spring-aop-3.2.13.RELEASE.jar:3.2.13.RELEASE]
370. at org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvocation.java:150) [spring-aop-3.2.13.RELEASE.jar:3.2.13.RELEASE]
371. at org.springframework.aop.aspectj.MethodInvocationProceedingJoinPoint.proceed(MethodInvocationProceedingJoinPoint.java:80) [spring-aop-3.2.13.RELEASE.jar:3.2.13.RELEASE]
372. at org.springframework.aop.aspectj.AbstractAspectJAdvice.invokeAdviceMethodWithBeforeAndAfter(AbstractAspectJAdvice.java:641) [spring-aop-3.2.13.RELEASE.jar:3.2.13.RELEASE]
373. at sun.reflect.GeneratedMethodAccessor689.invoke(Unknown Source) [:1.7.0_75]
374. at sun.reflect.DelegatingMethodAccessorImpl.invoke(DelegatingMethodAccessorImpl.java:43) [rt.jar:1.7.0_75]
375. at java.lang.reflect.Method.invoke(Method.java:606) [rt.jar:1.7.0_75]
376. at org.springframework.aop.aspectj.AbstractAspectJAdvice.invokeAdviceMethodWithBeforeAndAfter(AbstractAspectJAdvice.java:641) [spring-aop-3.2.13.RELEASE.jar:3.2.13.RELEASE]

```

StringBuilder进行append导致的内存溢出

| 内存区域       | 内存溢出的测试方法                                                                                                |                                                       |
|------------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| Java 堆     | 无限循环地 new 对象出来，在 List 中保存引用，以不被垃圾收集器回收。另外，该区域也有可能发生内存泄露（Memory Leak），出现问题时，要注意区别。                        |                                                       |
| 方法区        | 生成大量的动态类，或无限循环调用 String 的 intern() 方法产生不同的 String 对象实例，并在 List 中保存其引用，以不被垃圾收集器回收。后者测试常量池，前者测试方法区的非常量池部分。 |                                                       |
| 虚拟机栈和本地方法栈 | 单线程                                                                                                      | 多线程                                                   |
|            | 递归调用一个简单的方法：如不断累积的方法。会抛出 <u>StackOverflowError</u>                                                       | 无限循环地创建线程，并未每个线程无限循环地增加内存。会抛出 <u>OutOfMemoryError</u> |

# 小结

AI系统 & 深度学习系统: <https://chenzomi12.github.io/index.html>

## AI系统全栈架构图

